

SOFTWARE DEVELOPMENT PROCESSES

Lab Sheet 4 – Testing & Maintenance

INTRODUCTION

This week in lectures, we talked about several topics. For software testing, we reviewed a variety of different techniques used to verify the correctness of software under development. We looked at bug reports and bug tracking, and at common forms of software maintenance. To plan these activities for the scenario in the case study, you will need to refer back to the components you identified in your system architecture last week, so you will need to review your notes from last week's session.

TASK 1: CREATING A TESTING PLAN

Most complex systems cannot be completely tested using just one method, with different software components needing different forms of testing. For example, a server component that will be used by multiple clients should undergo load testing, while a UI component used by a single concurrent user will find it less useful. Developers plan the testing methods that they are going to use to test each individual system component fairly early, because they need to plan for the time and resources required to actually perform the testing.

Many development teams write up a *testing plan*, which details what types of testing will be used for each of the components in the system. Look back at your list of system components from last week's session, and discuss with the people around you which testing methods you think would be most suitable for each one, and write the notes up into a basic testing plan. For each testing method you select, make a note of one or two tests that would be performed on the system using this method.

TASK 2: DEVISING A MAINTENANCE SCHEDULE

Different system components will have vastly different maintenance needs. For example, an application installed on a desktop PC will likely only need attention if a software update breaks things or if the machine itself requires maintenance. Servers, on the other hand, often require much more regular maintenance: logs need rotating, out-of-date data needs removing, and other software on the machine needs patching (e.g. when a security flaw is found in a server OS).

Take another look at your list of software components. Which are likely to need regular maintenance attention? How often should it be performed? Can multiple components be maintained during a single slot of regularly-scheduled downtime? When should this be? Discuss your ideas with the people around you.

TASK 3: CLASS DISCUSSION

Your lab tutor will gather everyone together with time before the end of the lab session to go through your plans and schedules. Do people generally agree on what types of testing are going to be most important where? Are you surprised by just how much testing work there is likely to be? Do your maintenance schedules look suspiciously alike those of your favourite online games?

Keep a copy of your plans, they will be helpful in future weeks.